Distributed Shor's Algorithm with Sequential Quantum Teleportation

Jehn-Ruey Jiang
Department of Computer Science and Information Engineering
National Central University
Taoyuan City, Taiwan
jrjiang@csie.ncu.edu.tw

Ting-Yuan Wang
Department of Computer Science and Information Engineering
National Central University
Taoyuan City, Taiwan
pahamanx@g.ncu.edu.tw

Wen-Han Huang
Department of Computer Science and Information Engineering
National Central University
Taoyuan City, Taiwan
blade2556@g.ncu.edu.tw

Jie-Zhi Zhang
Department of Computer Science and Information Engineering
National Central University
Taoyuan City, Taiwan
george1130@g.ncu.edu.tw

2023 IEEE 5th Eurasia Conference on IOT, Communication and Engineering (ECICE) | 979-8-3503-1469-4/23/$31.00 ©2023 IEEE | DOI: 10.1109/ECICE59523.2023.10383099

***Abstract*—A distributed Shor's algorithm is used to factorize a semiprime with two quantum computers using quantum teleportation to transport quantum states of qubits in parallel. We propose another distributed Shor's algorithm with sequential quantum teleportation to improve the existing one. It explores how to employ distributed quantum computing through a quantum entanglement pair of qubits to transport quantum states of qubits sequentially. We combined two quantum computers to perform the quantum phase estimation (QPE) of Shor's algorithm. This reduced the required number of qubits for a single quantum computer, decreased quantum circuit depth, mitigated the impact of insufficient decoherence time, and accelerated the process of breaking cryptographic systems such as Rivest-Shamir-Adleman (RSA). The designed quantum circuit was executed on the IBM Quantum Lab platform to verify its correctness. We analyzed and compared the required numbers of qubits to run different versions of Shor's algorithm for individual quantum computers, as well as the numbers of qubits required to break different RSA cryptographic systems.**

Keywords—distributed quantum algorithm, quantum teleportation, RSA cyptography, Shor's algorithm, semiprime factorization

I. INTRODUCTION

Shor's algorithm [1], proposed by Peter Shor in 1994, is a pioneering quantum algorithm. It solves the semiprime factorization (SPF) and discrete logarithm (DL) problems with polynomial time complexity through collaborative computation between classical and quantum computers. A significant threat is widespread in public-key cryptographic systems such as Rivest-Shamir-Adleman (RSA), Diffie-Hellman (DH), and ElGamal. Their security relies on the computational difficulty of solving the SPF or DL problems. Current algorithms for solving these two problems operate on classical computers and require exponential time complexity. As a result, the aforementioned cryptographic systems are considered computationally secure with sufficiently long keys, such as RSA-2048, which uses a 2048-bit key. However, Shor's algorithm based on the computation power of quantum computers still poses a potential risk to the security of these systems.

Quantum computers harness quantum mechanical phenomena such as quantum superposition, entanglement, interference, and tunneling [2] to perform computation based on quantum bits (qubits). Their computational capacity grows exponentially with the number of qubits, in contrast to classical computers based on classical bits, which exhibit linear growth. This suggests the potential for exponential speedup of quantum computers over classical computers [3,4]. However, quantum computer development is challenging as it requires strict environmental conditions to manifest quantum properties. Thus, quantum computers usually have a limited number of qubits, short decoherence time, and low fidelity. For instance, the latest quantum computer, IBM Osprey, has only 433 qubits with unsatisfactory decoherence time and fidelity, leaving a gap in the practicality of breaking the RSA and other cryptographic systems.

We developed a distributed Shor's algorithm with sequential quantum teleportation to enhance the distributed Shor's algorithm introduced in [5]. We investigated the approach of distributed quantum computation through Einstein-Podolsky-Rosen (EPR) quantum entanglement pairing to achieve sequential quantum teleportation. This enables cooperative computation across multiple quantum computers for executing the quantum phase estimation (QPE) of Shor's algorithm. The developed distributed Shor's algorithm reduced the number of required qubits on a single quantum computer, decreased quantum circuit depth, mitigated the impact of limited quantum decoherence time, and accelerated the process of breaking public-key cryptographic systems such as RSA by using Shor's algorithm. We designed quantum circuits and ran them on the IBM Quantum Lab platform to experimentally validate the correctness of the proposed distributed Shor's algorithm. We also analyzed and compared the qubit requirements for different versions of Shor's algorithm on individual quantum computers and for breaking various RSA cryptographic systems.

The remainder of this article is organized as follows. Section II introduces Shor's algorithm, quantum teleportation, and distributed Shor's algorithm. Section III presents the proposed distributed Shor's algorithm with sequential quantum teleportation. Section IV demonstrates the implementation of quantum circuits for simulating the proposed algorithm. This section also shows the analysis of qubit requirements for different versions of Shor's algorithm. Finally, Section V concludes the entire article.

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 16:01:14 UTC from IEEE Xplore. Restrictions apply.

II. Shor's Algorithm and Distributed Shor's Algorithm

A. Shor's Algorithm

Given a semiprime $N = p \times q$, where p and q are two distinct prime numbers greater than 2, Shor's algorithm factorizes the integer N into the product of p and q. Shor's algorithm leverages the concept of Euler's theorem, which states that if integers a and N are coprime, then the modular exponential function $f(x) = a^x \pmod N$ is a periodic function. Specifically, Shor's algorithm uses quantum computation to efficiently find the period r of the modular exponential function $f(x)$ in polynomial time complexity. Subsequently, it employs classical computation, also in polynomial time complexity, to factorize the large integer N into the product of two prime numbers based on the obtained period r.

The following is the execution process of Shor's algorithm for factorizing a semiprime N into the product of p and q.

Step 1. Selecting an integer a that is less than N

Step 2. Using the Euclidean algorithm (i.e., the division algorithm) in classical computation to find gcd(a, N)

Step 3. If gcd(a, N) = $p \neq 1$, indicating the discovery of a non-trivial factor p of N and $q = N/p$ such that $N = p \times q$, then the algorithm outputs $N = p \times q$ and terminates.

Step 4. If gcd(a, N) = 1, then quantum phase estimation (QPE) is employed to determine the period r of the modular exponential function $f(x) = a^x \pmod N$.

Step 5. If r is odd or $a^{r/2} = -1 \pmod N$, then go to Step 1.

Step 6. Using the Euclidean algorithm in classical computation to find two integers, gcd($a^{r/2}+1$, N) and gcd($a^{r/2} - 1$, N). One of these two integers must be a non-trivial factor of N. Let this factor be p, and let the other factor be $q = N/p$. The algorithm outputs $N = p \times q$ and terminates.

As shown in Fig. 1, the QPE performed in Step 4 can be further broken down into the following sub-steps.

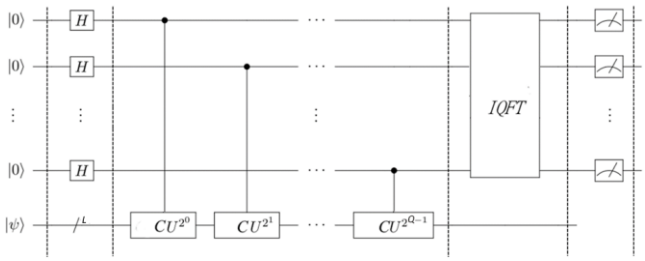


Fig. 1. Quantum circuit of quantum phase estimation (QPE) [6].

Step 4.1. Applying the Hadamard gate to Q counting qubits to form $|+\rangle^{\otimes Q}$ to be used later and preparing $|\psi\rangle = |0 \ldots 1\rangle$ for the L-qubit input x of the function $f(x) = a^x \pmod N$

Step 4.2. Preparing the unitary transformation U that is applied to the L qubits of x to compute $f(x) = a^x \pmod N$ to form the controlled unitary transformation CU later

Step 4.3. Applying controlled unitary transformations $CU^{2^0}, CU^{2^1}, \ldots, CU^{2^{Q-1}}$ to the L qubits of x with one of Q counting qubits being the control qubit of one CU. The inverse quantum Fourier transform denoted as IQFT or QFT^{-1} or $QFT^{\ddagger}$, is then applied to the Q counting qubits to determine $2^Q\lambda$, where λ is the phase associated with the eigenvalue of the unitary transformation U.

Step 4.4. Measuring to obtain every significant result $m = 2^Q\lambda$ with a significant occurrence probability

Step 4.5. For every significant measurement result m, $\lambda = m/2^Q$ is calculating to represent it as $m/2^Q = s/r$ with continued fraction expansion, where s and r are both integers and $0 \leq s \leq r-1$. The integer r derived from one of the significant measurement results is the period of function $f(x) = a^x \pmod N$ with a very high probability.

In Step 4.1, $L = \lceil log\, N \rceil$ is the number of qubits used to represent integer N. The L qubits serve as the input qubits of controlled unitary transformations $CU^{2^0}, CU^{2^1}, \ldots, CU^{2^{Q-1}}$; they are called computing qubits. Furthermore, there are Q counting qubits prepared in Step 4.1. The Q counting qubits are also called control qubits, as they serve as control qubits of the controlled unitary transformations. As Q increases, the precision in determining the integer period r improves. Typically, we adopt the setting $Q = 2L + 1$, ensuring a high probability of accurately determining the period r.

The derivation for $Q = 2L+1$ is shown in (1) and (2). Shor's algorithm employs L computing bits and Q counting bits to perform QPE, to determine the phase λ associated with the eigenvalue $e^{2\pi i\lambda}$ of the modular exponential function $f(x) = a^x \pmod N$, where $0 \leq \lambda < 1$. After deriving λ, Shor's algorithm derives the period r of $f(x)$ by using continued fraction expansion to approximate λ as s/r, where s and r are integers, and $0 \leq s < r$. Since the number of counting qubits Q is finite, Q counting qubits can only yield an approximate value $\hat{\lambda}$ for λ. According to Theorem 5.1 in Ref. [7], to ensure that the continued fraction expansion s/r based on $\hat{\lambda}$ converges and approximates the true λ, the following equation must be satisfied.

$$\left|\frac{s}{r} - \hat{\lambda}\right| \leq \frac{1}{2r^2} \tag{1}$$

If we employ at least $Q = 2L+1 = 2\lceil log\, N \rceil + 1$ counting qubits to represent $\hat{\lambda}$, then Eq. (1) holds. The rationale for this is as follows.

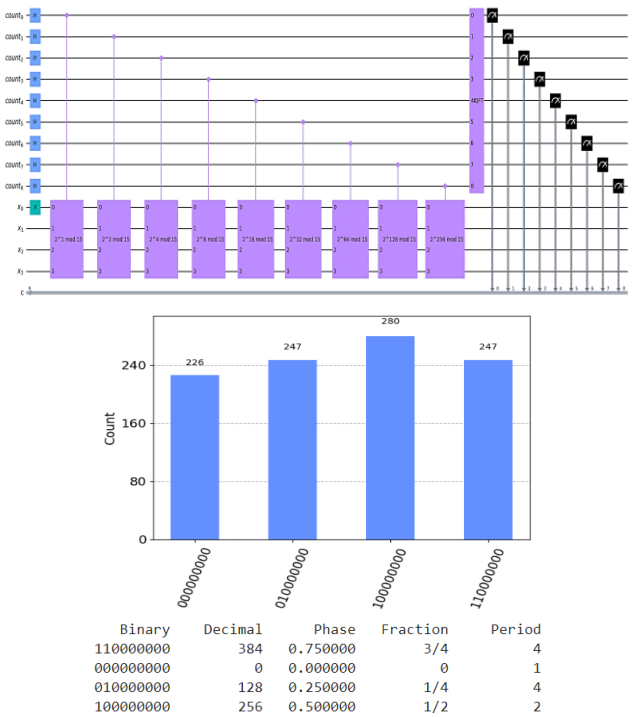
$$\left|\frac{s}{r} - \hat{\lambda}\right| \leq \frac{1}{2^{2\lceil logN \rceil + 1}} = \frac{1}{2(2\lceil logN \rceil)^2} \leq \frac{1}{2N^2} \leq \frac{1}{2r^2} \tag{2}$$

In contrast to the derivation of counting qubit requirements mentioned above, some studies [8−11] approach the derivation from the perspective of implementing QPE in Shor's algorithm. They indicate that Shor's algorithm requires $2L+3$, $2L+2$, $2L+2$, and $2L+1$ qubits for QPE implementation. These studies focused on how to efficiently utilize auxiliary qubits to realize operations related to Shor's algorithm QPE, such as the modular exponential function and the IQFT, rather than the requirement of counting qubits.

We temporarily set aside considerations regarding qubit requirements stemming from QPE implementation details. Overall, without considering implementation details, we assumed that Shor's algorithm required $3L+1 = 3\lceil log\, N \rceil + 1$ qubits for implementing QPE, where L qubits are computing qubits and $Q = 2L+1$ qubits are counting qubits or control qubits. For example, Fig. 2 illustrates a quantum circuit using

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 16:01:14 UTC from IEEE Xplore. Restrictions apply.

13 qubits to realize Shor's algorithm QPE for $f(x) = a^x \pmod N = 2^x \pmod{15}$. The quantum circuit was executed on the IBM Quantum Lab platform for obtaining the measurement results and deriving the period r of $f(x)$, which are also shown in Fig. 2. The derived period r is 4, which was employed by Shor's algorithm in Step 6 to determine $\gcd(a^{r/2}+1, N) = \gcd(2^{4/2}+1, 15)=5$, and $\gcd(a^{r/2}-1, N) = \gcd(2^{4/2}-1, 15) = 3$. This indeed successfully factors integer N=15 into 3×5.



Binary	Decimal	Phase	Fraction	Period
110000000	384	0.750000	3/4	4
000000000	0	0.000000	0	1
010000000	128	0.250000	1/4	4
100000000	256	0.500000	1/2	2

Fig. 2. Quantum circuit realizing quantum phase estimation (QPE) of Shor's algorithm, its measurement results, and the derivation of the period of the modular exponential function.

B. Distributed Shor's Algorithm

A distributed Shor's algorithm was proposed [5] to perform distributed quantum computation in a distributed quantum computing system (DQCS) to accomplish the QPE of Shor's algorithm. DQCS refers to a system composed of numerous finite-capacity quantum computers interconnected through classical and quantum channels, forming a quantum network or quantum Internet. Within DQCS, the qubits in each quantum computer interact with other qubits in the same quantum computer for quantum computation. Moreover, the states of qubits in one quantum computer are transmitted to qubits in another quantum computer through quantum teleportation for distributed quantum computation interactions. Qubits in different quantum computers interact with each other for distributed quantum computation interactions through remote or non-local quantum gates [12]. The algorithm proposed in Ref. [5] accomplishes distributed quantum computation by transmitting qubit states between different quantum computers using quantum teleportation.

Figure 3 illustrates the architectural diagram of the distributed Shor's algorithm proposed in [5]. This architecture employs two quantum computers, where the left side represents quantum computer A, and the right side represents quantum computer B. In Fig. 3, M_a represents $CU^{2^0}, CU^{2^1}, \ldots, CU^{2^{(L/2)+1}}$ and $M_a^{2^{(L/2)}}$ represents $CU^{2^{(L/2)}}, CU^{2^{(L/2)+1}}, \ldots, CU^{2^{2L+1}}$, as elaborated below.

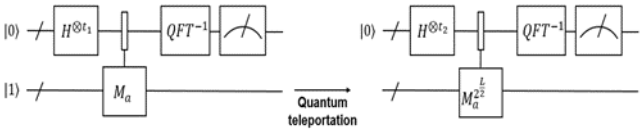


Fig. 3. Architectural diagram of the distributed Shor's algorithm [5] (the figure is adapted from Ref. [5]).

The distributed Shor's algorithm employs quantum computer A (QC A) and quantum computer B (QC B) to jointly perform (or compute) QPE using $2L$+1 counting qubits. Initially, QC A handles QPE from the 1st to the $((L/2)+1)^{th}$ counting qubits. These counting qubits or control qubits respectively control the controlled unitary transformations (CUs), namely $CU^{2^0}, CU^{2^1}, \ldots$, and so forth. Moreover, after undergoing the IQFT (or QFT^{-1}), QC A completes the QPE computation for the first $(L/2)+1$ counting qubits. Subsequently, QC A employs quantum teleportation to transmit the quantum states of the computing qubits of the CUs to QC B. Afterwards, QC B continues to compute QPE from the $(L/2)^{th}$ to the $(2L+1)^{th}$ counting qubits. Similarly, after performing IQFT, QC B completes the QPE computation for the last $(3L/2)+2$ counting qubits.

QC A and QC B duplicate the QPE computation for the $(L/2)^{th}$ and the $((L/2)+1)^{th}$ counting qubits. The duplication is to merge the measurement outcomes of QC A and QC B to increase the probability of correct period finding for the modular exponential function. The outcome merging can be performed efficiently by classical computers [5]. The following are the steps of the distributed Shor's algorithm proposed in Ref. [5].

Step 1. Initially, QC A prepares $t_1 = (L/2)+1$ counting qubits for QPE and L computing qubits for CUs. QC B prepares $t_2 = (3L/2)+2$ counting qubits for QPE and L computing qubits for CUs. Both quantum computers also jointly prepare L EPR pairs of entangled qubits, each individually owning one qubit from each pair for quantum teleportation purposes. Therefore, each quantum computer possesses L quantum teleportation qubits (QT qubits) for transporting the states of L computing qubits in parallel.

Step 2. QC A performs QPE for the first t_1 counting qubits and then performs IQFT. Afterward, it transports the quantum states of the L computing qubits through the L QT qubits to QC B with parallel quantum teleportation for continued computation of QPE.

Step 3. Upon receiving the quantum states of the L computing qubits from QC A, QC B performs QPE for the last t_2 counting qubits. It then also performs IQFT.

Step 4. QC A and QC B individually measure t_1 and t_2 counting qubits, resulting in outcomes m_1 and m_2, respectively. Note that to maintain high precision in determining the period r, QC A and QC B duplicate the QPE computation for the $(L/2)^{th}$ and the $((L/2)+1)^{th}$ counting qubits.

Step 5. Classical computation is employed to compare the duplicated part of measurement outcomes m_1 and m_2 for generating the merged result m. Finally, the algorithm reverts to the original Shor's algorithm to determine the period r of the modular exponentiation function $f(x) = a^x \pmod N$.

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 16:01:14 UTC from IEEE Xplore. Restrictions apply.

III. Distributed Shor's Algorithm with Sequential Quantum Teleportation

The distributed Shor's algorithm with sequential quantum teleportation was developed in this study. The steps of the proposed algorithm are fundamentally similar to those of the distributed Shor's algorithm proposed in [5]. Our propsed distributed Shor's algorithm uses only one EPR pair of entangled qubits and performs quantum teleportation for L times to sequentially transport the quantum states of a total of L computing qubits. Contrarily, the distributed Shor's algorithm proposed in [5] uses L EPR pairs of entangled qubits and performs quantum teleportation to transport the quantum states of L computing qubits in parallel at the same time. The steps of our proposed algorithm are as follows.

Step 1. Initially, quantum computer A' (QC A') prepares $t_1=(L/2)+1$ counting qubits for QPE and L computing qubits for CUs. Quantum computer B' (QC B') prepares $t_2=(3L/2)+2$ counting qubits for QPE and L computing qubits for CUs. Both quantum computers also jointly prepare one EPR pair of entangled qubits, each individually owning one qubit from the pair for quantum teleportation purposes. Therefore, each quantum computer possesses one QT qubit for transporting the states of L computing qubits sequentially.

Step 2. QC A' performs QPE for the first t_1 counting qubits, and then performs IQFT. Afterward, it transports the quantum states of the L computing qubits through the unique QT qubit to QC B' with sequential quantum teleportation for continued computation of QPE.

Step 3. Upon receiving the quantum states of the L computing qubits from QC A', QC B' performs QPE for the last t_2 counting qubits. It then also performs IQFT.

Step 4. QC A' and QC B' individually measure t_1 and t_2 counting qubits, resulting in outcomes m_1 and m_2, respectively. To maintain high precision in determining the period r, the QPE computation for the $(L/2)^{th}$ and the $((L/2)+1)^{th}$ counting qubits duplicates on both QC A' and QC B'.

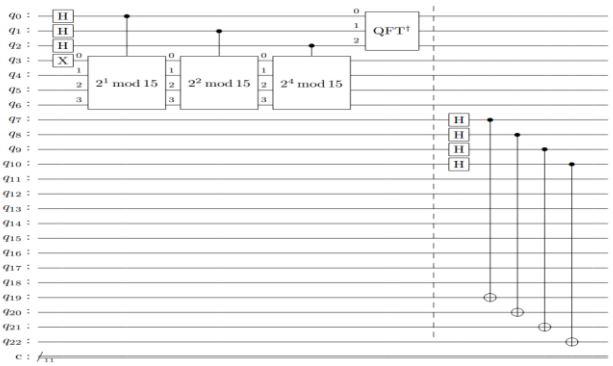
Step 5. Classical computation is employed to compare the duplicated part of measurement outcomes m_1 and m_2 for generating the merged result m. Finally, the algorithm reverts to the original Shor's algorithm to determine the period r of the modular exponentiation function $f(x) = a^x \pmod N$.

IV. Experimental Results and Performance Comparisons

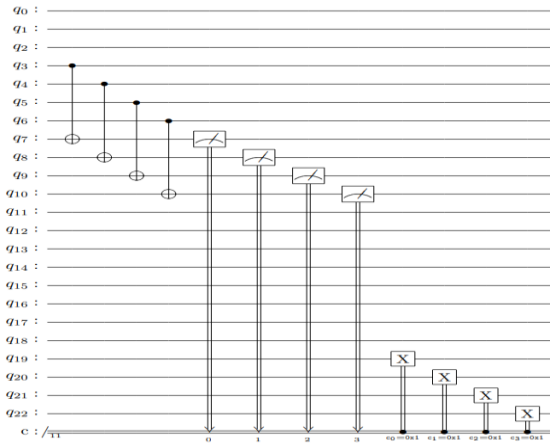
A. Experimental Results

We experimented with the IBM Quantum Lab platform to simulate both the distributed Shor's algorithm [5] and the distributed Shor's algorithm with sequential quantum teleportation proposed in this study. The experiments were performed for factorizing the semiprime N=15, with the parameter settings of $L = \lceil log\ N \rceil = 4$, and Q=2L+1=9. To clarify, we utilized 4 computing qubits and 9 counting qubits. Including 2 duplicate counting qubits, there were a total of 11 counting qubits used in the implementation of the algorithms. Finally, through merging measurement outcomes, we accurately determined the period r=4 for the modular exponentiation function $f(x) = a^x \pmod N = 2^x \pmod{15}$. In the course of our experiments, both quantum computers had identical results of duplicate counting qubits. Therefore, during the merging process, it was straightforward to remove duplicate counting qubits.

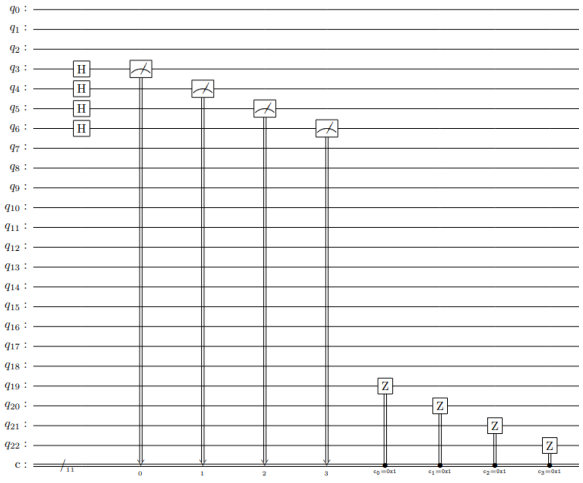
Figure 4 depicts the quantum circuit for the distributed Shor's algorithm [5] using two quantum computers. Fig. 4(a) represents QC A performing QPE for counting qubits 1−3 and preparing four EPR pairs of entangled qubits. Figure 4(b) illustrates QC A initiating quantum teleportation to transport quantum states of its four computing qubits. Figure 4(c) demonstrates QC A continuing the quantum teleportation process for QC B to properly receive the quantum states of four computing qubits of QC A. Figure 4(d) portrays QC B performing QPE for the counting qubits 2-9. Finally, Fig. 4(e) represents both QC A and QC B conducting counting qubit measurements to be merged on a classical computer for finding the period of the modular exponential function.



(a) QC A performing QPE for the counting qubits 1-3 and preparing four EPR pairs of entangled qubits

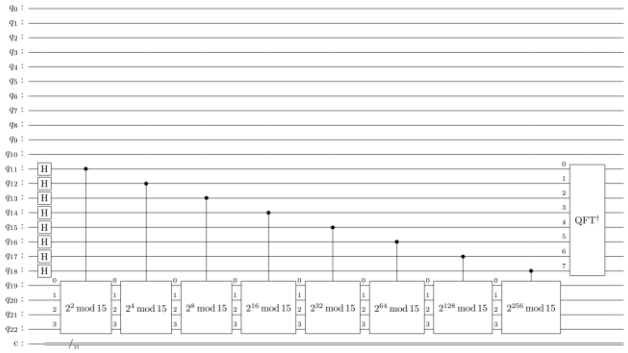


(b) QC A initiating quantum teleportation to transport quantum states of four computing qubits

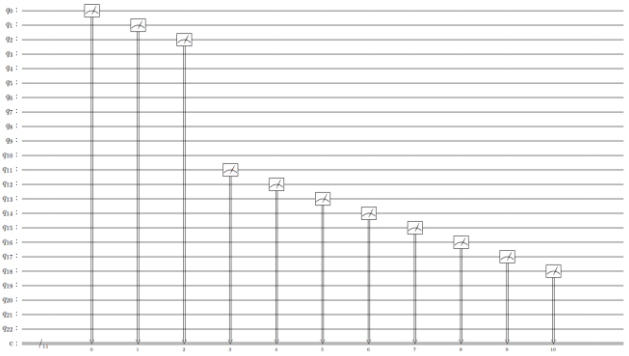


(c) QC A continuing quantum teleportation for QC B to receive the quantum states of four computing qubits of QC A

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 16:01:14 UTC from IEEE Xplore. Restrictions apply.



(d) QC B performing QPE for the counting qubits 2-9

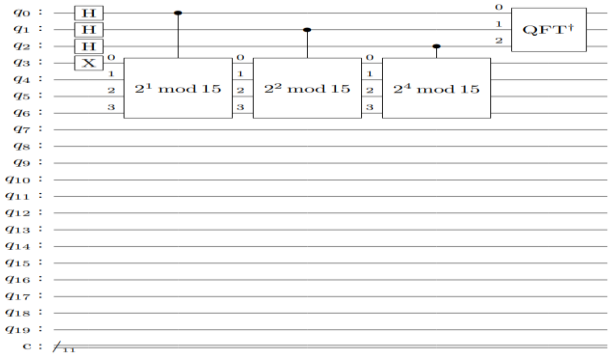


(e) QC A and QC B conducting counting qubit measurements to be merged

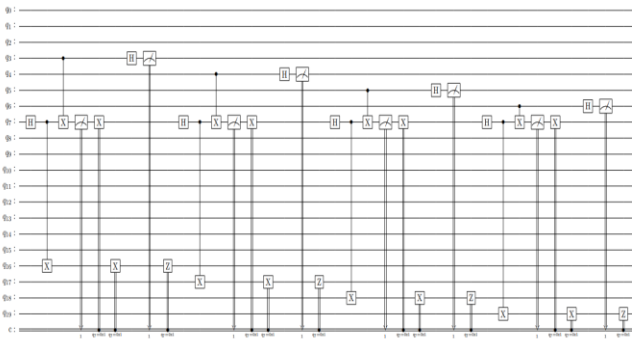
Fig. 4. Quantum circuit realizing quantum phase estimation (QPE) of the distributed Shor's algorithm.

In the quantum circuit of the distributed Shor's algorithm, QC A used 11 qubits, whereas QC B used 12 qubits. This indicated a reduction of one qubit in the maximum qubit requirement for a single quantum computer, compared to the original Shor's algorithm, which requires 13 qubits on a single quantum computer. We will subsequently analyze the qubit requirements for each quantum computer in the distributed Shor's algorithm for tasks such as the factorization of different integers and the decryption of RSA cryptographic systems.

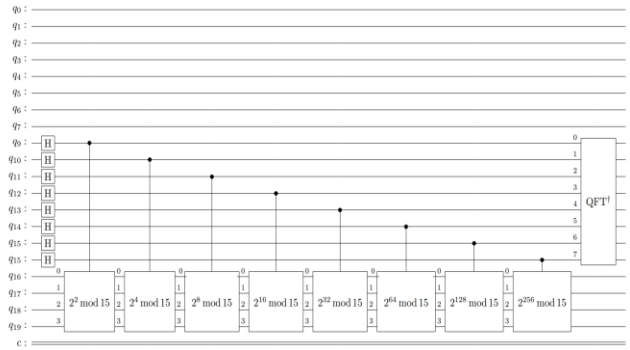
Figure 5 shows the quantum circuit of the distributed Shor's algorithm with sequential quantum teleportation using two quantum computers, QC A' and QC B'. Figure 5(a) represents QC A' performing QPE for the counting qubits 1-3. Figure 5(b) illustrates QC A' transporting quantum states of its four computing qubits to QC B' through sequential quantum teleportation. Figure 5(c) portrays QC B' performing QPE for the counting qubits 2-9. Finally, Fig. 5(d) represents both QC A' and QC B' conducting counting qubit measurements to be merged on a classical computer for finding the period of the modular exponential function.



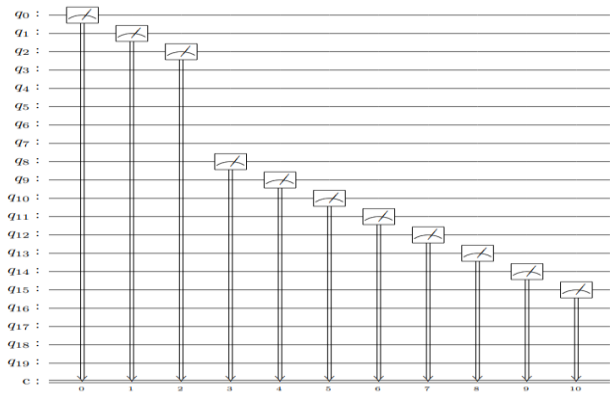
(a) QC A' performing QPE for the counting qubits 1-3



(b) QC A' transporting quantum states of its four computing qubits to QC B' through sequential quantum teleportation



(c) QC B' performing QPE for the counting qubits 2-9



(d) QC A' and QC B' conducting counting qubit measurements

Fig. 5. Quantum circuit realizing quantum phase estimation (QPE) of the distributed Shor's algorithm with sequential quantum teleportation.

B. *Performance Analysis and Comparisons*

We compared the performance of different versions of Shor's algorithm. The original Shor's algorithm requires a total of $3L+1$ qubits. In the distributed Shor's algorithm [5], quantum computer A (QC A) requires a total of $(L/2)+1+L+L=(5L/2)+1$ qubits, whereas quantum computer B (QC B) requires a total of $(3L/2)+2+L=(5L/2)+2$ qubits. This reduces $L/2$ qubits in the minimum qubit requirement for a single quantum computer compared to the original Shor's algorithm, which requires $3L+1$ qubits. Additionally, apart from the reduction in minimum qubit requirement for a single quantum computer, the depth of the quantum circuit can also be reduced.

In the proposed distributed Shor's algorithm with sequential quantum teleportation, quantum computer A' (QC A') requires a total of $(L/2)+1+L+1 = (3L/2)+2$ qubits, whereas quantum computer B' (QC B') requires a total of $(3L/2)+2+L=(5L/2)+2$ qubits. This further reduces the minimum qubit requirement for a single quantum computer by $(3L/2)-1$ qubits

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 16:01:14 UTC from IEEE Xplore. Restrictions apply.

when compared to the original Shor's algorithm, which requires $3L+1$ qubits. However, since the proposed algorithm adopts sequential quantum teleportation to transport quantum states of L computing qubits one qubit by one qubit, the time of the total quantum teleportation process increases, and the quantum circuit depth is lengthened when compared with the distributed Shor's algorithm [5], which uses parallel quantum teleportation to transport quantum states. Nevertheless, the proposed algorithm reduces the difficulty of the simultaneous generation and distribution of L EPR pairs of entangled qubits and further reduces the minimum qubit requirement for a single quantum computer to $(3L/2)+2$ qubits.

Table I sshows the comparison of the performance of different Shor's algorithms, namely, the original Shor's algorithm (SA), distributed Shor's algorithm (DSA), and distributed Shor's algorithm with sequential quantum teleportation (DSA/SQT), in terms of the qubit requirement per quantum computer. The contents in a table cell are separated by commas for QC A on the left and QC B on the right, or for QC A' on the left and QC B' on the right. Figure 6 shows the qubit requirement of different Shor's algorithms for factorizing N=16 and for breaking RSA-512, RSA-1024, and RSA-2048 cryptographic systems. The proposed DSA/SQT algorithm significantly reduces the minimum qubit requirement per quantum computer, with greater reductions as the key lengths of the cryptographic systems increase, as shown in Fig. 7.

TABLE I. COMPARISONS OF VARIOUS VERSIONS OF SHOR'S ALGORITHMS IN TERMS OF QUBIT REQUIREMENTS

Algorithms	SA	DSA	DSA/SQT
Computing Qubits	L	L, L	L, L
Counting Qubits	$2L+1$	$(L/2)+1$, $(3L/2)+2$	$(L/2)+1$, $(3L/2)+2$
QT Qubits	0	L, 0	1, 0
Total Qubits per QC	$3L+1$	$(5L/2)+1$, $(5L/2)+2$	$(3L/2)+2$, $(5L/2)+2$

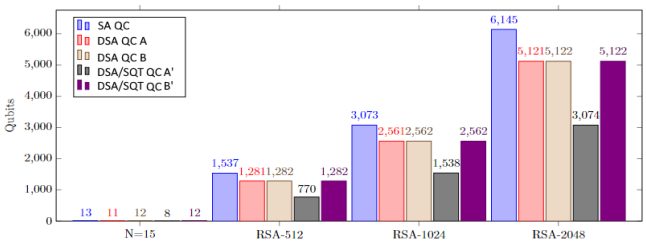


Fig. 6. Qubit requirement per quantum computer for different Shor's algorithms when factorizing N=16 and breaking various RSA systems.



Fig. 7. Maximum reduction in the qubit requirement per quantum computer when factorizing N=16 and breaking various RSA systems, as achieved by different distributed Shor's algorithms.

ACKNOWLEDGMENT

We express our gratitude to the "National Taiwan University - IBM Quantum Computing Center (IBM Q Hub at NTU)" for providing us with access to the IBM Q system.

REFERENCES

[1] P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," In Proc. of 35th annual symposium on foundations of computer science, pp. 124-134, 1994.

[2] J.-R. Jiang, and C.-W. Chu, "classifying and benchmarking quantum annealing algorithms based on quadratic unconstrained binary optimization for solving NP-hard problems," IEEE Access, Vol. 11, pp. 104165-104178, 2023.

[3] J.-R. Jiang, "Quantum circuit based on Grover algorithm to solve hamiltonian cycle problem," in Proc. of the IEEE 4th Eurasia Conference on IOT, Communication and Engineering (IEEE ECICE 2022), Oct. 2022.

[4] J.-R. Jiang, and Y.-J. Wang, "Quantum circuit based on Grover's algorithm to solve exact cover problem," in Proc. of the 19th VTS Asia Pacific Wireless Communications Symposium (APWCS 2023), Taiwan, 2023.

[5] L. Xiao, D. Qiu, L. Luo, and P. Mateus, "Distributed Shor's algorithm," Quantum Information and Computation, Vol. 23, No. 1&2, pp. 27–44, 2023.

[6] J.-R. Jiang, Easy to Learn Quantum Programming (in Chinese), GoTop Publishing Co., 2022.

[7] M. A. Nielsen, and I. L. Chuang, Quantum computation and quantum information. Cambridge University Press, 2010.

[8] S. Beauregard, "Circuit for Shor's algorithm using 2n+3 qubits," arXiv preprint quant-ph/0205095, 2002.

[9] Y. Takahashi, and N. Kunihiro, "A quantum circuit for Shor's factoring algorithm using 2n+2 qubits," Quantum Information & Computation, 6(2):184–192, 2006.

[10] T. Häner, M. Roetteler, and K, M. Svore, "Factoring using 2n+2 qubits with Toffoli based modular multiplication," arXiv preprint arXiv:1611.07995, 2016.

[11] C. Gidney, "Factoring with n+2 clean qubits and n-1 dirty qubits," arXiv preprint arXiv:1706.07884, 2017.

[12] A. Yimsiriwattana, and S. J. Lomonaco Jr, "Distributed quantum computing: A distributed Shor algorithm," Quantum Information and Computation II, Vol. 5436, pp. 360-372, 2004.

Authorized licensed use limited to: Integral University - Lucknow. Downloaded on August 03,2026 at 16:01:14 UTC from IEEE Xplore. Restrictions apply.